Understanding and Implementation Strategy for RV-Sparse

A RISC-V Vector Accelerated Sparse Linear Algebra Library

LFX Summer 2026 Proposal

Muhammad Waleed Akram

Electrical Engineering Student
University of Engineering and Technology (UET), Lahore

GitHub: Waleed99i

Hashnode: @waleed07

May 2026

Abstract

Sparse linear algebra plays an important role in modern computing applications such as scientific simulations, machine learning, graph analytics, and high-performance computing systems. Efficient sparse computation is challenging because sparse matrices involve irregular memory access patterns, indirect indexing, and non-uniform traversal behavior. The RV-Sparse project aims to develop an open-source sparse linear algebra library optimized for the RISC-V Vector Extension (RVV), enabling accelerator-independent sparse computation on general-purpose RISC-V systems.

This document presents my understanding of sparse matrix computation, CSR/CSC sparse representations, sparse traversal challenges, and the overall objectives of the RV-Sparse project. It also includes my coding challenge implementation understanding, proposed implementation strategy, contribution workflow, development timeline, and expected deliverables for the project.

Contents

1 Introduction

Sparse linear algebra is an important area in modern computing systems because many real-world datasets and computational workloads naturally contain a large number of zero-valued elements. Such sparse structures commonly appear in scientific simulations, graph analytics, recommendation systems, numerical solvers, and machine learning workloads.

In modern machine learning applications, especially pruned neural networks, sparsity is intentionally introduced to reduce:

- memory usage
- computational complexity
- bandwidth overhead

However, storing sparse matrices in standard dense form is highly inefficient because memory and computation are wasted on zero-valued elements.

To address this problem, sparse matrix representations such as:

- Compressed Sparse Row (CSR)
- Compressed Sparse Column (CSC)

store only the useful non-zero elements along with their structural information.

Efficient sparse computation is not only a mathematical problem but also a systems and computer architecture problem. Sparse workloads involve:

- irregular memory access patterns
- indirect indexing
- non-uniform traversal behavior
- cache locality challenges

These characteristics make sparse computation significantly more difficult to optimize compared to dense linear algebra operations.

The RV-Sparse project focuses on developing an open-source sparse linear algebra library optimized for the RISC-V Vector Extension (RVV). Instead of depending entirely on specialized sparse hardware accelerators, the project aims to enable efficient sparse computation on general-purpose RISC-V vector processors through software-level optimization and vectorized sparse kernels.

This document presents my understanding of sparse matrix computation, CSR/CSC representations, sparse traversal challenges, and the RV-Sparse project itself. It also outlines my implementation strategy understanding, proposed contribution workflow, timeline, and expected deliverables.

2 Sparse Matrix Computation Background

Sparse matrices are matrices in which most of the elements are zero-valued. Such matrices appear naturally in many modern computational workloads including:

- scientific computing
- machine learning
- graph analytics
- numerical simulations
- recommendation systems

In many practical applications, storing sparse matrices in dense form becomes inefficient because memory and computation are wasted on zero-valued elements.

To address this problem, sparse matrix representations store only the useful non-zero values along with their structural information.

2.1 Compressed Sparse Row (CSR)

One of the most widely used sparse formats is the Compressed Sparse Row (CSR) representation.

CSR stores sparse matrices using three arrays:

- `values[]`
- `col_indices[]`
- `row_ptrs[]`

The `values[]` array stores all non-zero values, while `col_indices[]` stores the corresponding column indices. The `row_ptrs[]` array stores the starting index of each row inside the CSR structure.

This representation significantly reduces:

- memory usage
- bandwidth overhead
- unnecessary computation

especially for highly sparse datasets.

2.2 Compressed Sparse Column (CSC)

Compressed Sparse Column (CSC) is another important sparse representation format. CSC is conceptually similar to CSR, but traversal occurs column-wise instead of row-wise.

CSC is particularly useful for:

- column-oriented sparse operations
- transpose-based computation
- sparse matrix factorization algorithms

2.3 Sparse Matrix-Vector Multiplication

One of the most fundamental sparse operations is Sparse Matrix-Vector Multiplication (SpMV):

$$y = A \times x$$

where:

- A is a sparse matrix
- x is the input vector
- y is the output vector

Unlike dense matrix multiplication, sparse multiplication skips zero-valued elements and operates only on non-zero entries. This reduces computational overhead significantly.

2.4 Relation to RV-Sparse

The RV-Sparse project focuses on improving sparse linear algebra support using the RISC-V Vector Extension (RVV). Efficient sparse traversal combined with vectorized execution can help improve sparse computation performance on general-purpose RISC-V systems.

As part of my learning process, I explored sparse matrix computation concepts and documented them in detail through technical blog articles.

Hashnode Profile:

https://hashnode.com/@waleed07

Related Blog Articles:

- Understanding Sparse Matrix Computation and CSR Representation
- Explaining My RV-Sparse Coding Challenge Implementation in C

Project Understanding

3 Background

A relevant introduction to sparse computation concepts and motivation can be found in: https://youtu.be/Lhef_jxzqCg?si=SxfNV5YKVYKQJ8-m

There were long videos as well which I watched but above one is the most concised and good one.

4 Problem Statement

While dense linear algebra libraries such as OpenBLAS are highly optimized for general-purpose workloads, equivalent support for sparse matrix operations optimized for the RISC-V Vector Extension (RVV) remains limited.

This creates a performance gap when executing sparse workloads on RISC-V-based systems. The challenge is to design lightweight, efficient software solutions that:

- Avoid unnecessary computation on zero-valued elements
- Minimize memory overhead
- Exploit data-level parallelism using vector-friendly layouts

5 Project Objective

The objective of the RV-Sparse project is to develop a lightweight C-based library for sparse linear algebra, focusing on:

- Efficient sparse matrix-vector multiplication
- Support for CSR and CSC formats
- RVV-friendly memory access patterns
- Clean and reusable API design

6 Scope of Work

The scope of this project includes:

- Implementation of sparse matrix-vector multiplication (SpMV)
- Handling of pre-allocated memory (no dynamic allocation inside core kernels)
- Optimized traversal strategies for sparse matrices
- Foundation for future RISC-V Vector (RVV) acceleration

Implementation Strategy

7 Overview

The official repository for this project is: https://github.com/merledu/rv-sparse

The implementation strategy is designed to progressively build a clean software foundation that can later be extended into fully vectorized RVV kernels.

8 Design Goals

The core design philosophy of the project is based on the following objectives:

- **Sparse-first computation:** Avoid unnecessary operations on zero elements.
- **Hardware awareness:** Prepare data structures that map efficiently to RISC-V Vector ISA.
- **Lightweight implementation:** Keep the library minimal, dependency-free, and portable in C.
- **Extensibility:** Ensure easy transition from scalar reference code to vectorized RVV kernels.

9 Implementation Strategy

The implementation follows a phased development approach:

9.1 Phase 1: Baseline Sparse Kernels

The first step focuses on scalar reference implementations of:

- Sparse Matrix-Vector Multiplication (SpMV)
- Sparse Matrix-Matrix Multiplication (SpMM) (planned/optional)

These implementations ensure correctness and act as a performance baseline.

9.2 Phase 2: Memory-Efficient Execution Model

A key constraint in the project is minimizing runtime memory overhead. Therefore:

- All buffers are externally allocated by the caller
- No dynamic allocation occurs inside compute kernels
- Computation is performed in-place wherever possible

This design improves predictability and aligns with embedded system requirements.

9.3 Phase 3: Vectorization Preparation

Before introducing RVV intrinsics, the scalar implementation is structured to:

- Maintain contiguous memory access patterns
- Reduce branching inside inner loops
- Isolate compute kernels for easy vector replacement

This ensures that future vectorized versions require minimal structural changes.

10 Project-Level Architecture

At a high level, the repository is structured around:

- Core sparse kernel implementations (CSR/CSC based)
- Utility functions for matrix generation and testing etc

11 Expected Evolution Path

The long-term goal of the project is to evolve from:

$$\text{Scalar Sparse Kernels} \rightarrow \text{Optimized Memory Layouts} \rightarrow \text{RVV Vectorized Kernels}$$

This progression ensures correctness-first development followed by performance optimization.

12 Summary

The RV-Sparse implementation strategy is designed as a structured progression from simple scalar sparse operations to highly optimized RISC-V vectorized kernels. The emphasis is on correctness, memory efficiency, and future hardware acceleration compatibility.

13 Weekly Timeline (Tabular View)

14 Expected Deliverables

The RV-Sparse project is expected to produce the following key deliverables over the course of development:

- A lightweight and modular C-based sparse linear algebra library.
- Efficient implementations of Sparse Matrix-Vector Multiplication (SpMV) using CSR format.

Week	Focus Area	Key Deliverables
Week 1	Sparse matrix fundamentals	Understanding CSR/CSC formats and sparse computation basics
Week 2	Environment setup and preprocessing	Development setup, initial dense-to-sparse conversion logic
Week 3	Baseline SpMV implementation	First working sparse matrix-vector multiplication with correctness validation
Week 4	CSR refinement	Optimized CSR traversal and reduced redundant operations
Week 5	CSC exploration	CSC implementation and CSR vs CSC comparative understanding
Week 6	Kernel modularization	Separation of computation and data structure logic for extensibility
Week 7	RVV introduction	Study of RISC-V Vector Extension and mapping loops to vector-friendly design
Week 8	Vectorization experiments	Identification of bottlenecks and baseline performance benchmarking
Week 9	Memory optimization	Improved cache-friendly access patterns and reduced branching
Week 10	Integration phase	Unified sparse API with stable kernel integration and testing
Week 11	Benchmarking	Performance evaluation against baseline scalar implementation
Week 12	Finalization	Code cleanup, documentation, and submission preparation

Table 1: 12-Week Implementation Timeline for RV-Sparse Project

- Support for Compressed Sparse Row (CSR) and Compressed Sparse Column (CSC) representations.
- Clean and well-documented API for sparse matrix operations.
- Optimized scalar baseline implementations prepared for RISC-V Vector Extension (RVV) acceleration.
- Benchmarking framework to evaluate performance across different matrix sizes and sparsity levels.
- Refactored and extensible codebase suitable for future vectorized RVV intrinsics integration.
- Complete documentation including usage examples and design explanations.

15 Conclusion

This project proposes a structured approach to implementing efficient sparse linear algebra operations for RISC-V Vector Extension enabled systems. It focuses on developing a lightweight and extensible library with optimized CSR/CSC-based computations. The work establishes a strong foundation for future vectorized implementations and performance improvements. Overall, it aims to bridge the gap between sparse workloads and efficient execution on general-purpose RISC-V platforms.